ComplaintsActivity.java

Esta clase se encarga de gestionar la funcionalidad de envío de quejas o sugerencias por parte de los residentes. Su objetivo es permitir que el usuario redacte un comentario y lo envíe directamente a la base de datos de Firestore, guardando automáticamente su correo electrónico y el momento exacto del reporte. Esto contribuye a mejorar la comunicación con la administración y asegura que las inquietudes de los vecinos queden registradas de manera formal para su posterior revisión, siguiendo un flujo de datos directo desde la interfaz hacia la colección de la nube.

CODIGO:

```
public class ComplaintsActivity extends AppCompatActivity {

    private EditText editTextQueja;

    private FirebaseFirestore db;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_complaints);

        db = FirebaseFirestore.getInstance();

        editTextQueja = findViewById(R.id.editTextQueja);

        findViewById(R.id.buttonBack).setOnClickListener(v -> finish());

        findViewById(R.id.buttonSendQueja).setOnClickListener(v -> {

            String queja = editTextQueja.getText().toString().trim();

            if (!queja.isEmpty()) sendQuejaToFirestore(queja);

            else Toast.makeText(this, "Escribe tu comentario",
Toast.LENGTH_SHORT).show();

        });

    }

    private void sendQuejaToFirestore(String mensaje) {

        String email = FirebaseAuth.getInstance().getCurrentUser() != null ?

                    FirebaseAuth.getInstance().getCurrentUser().getEmail() :
"anonimo";

        Map<String, Object> data = new HashMap<>();

        data.put("mensaje", mensaje);
```

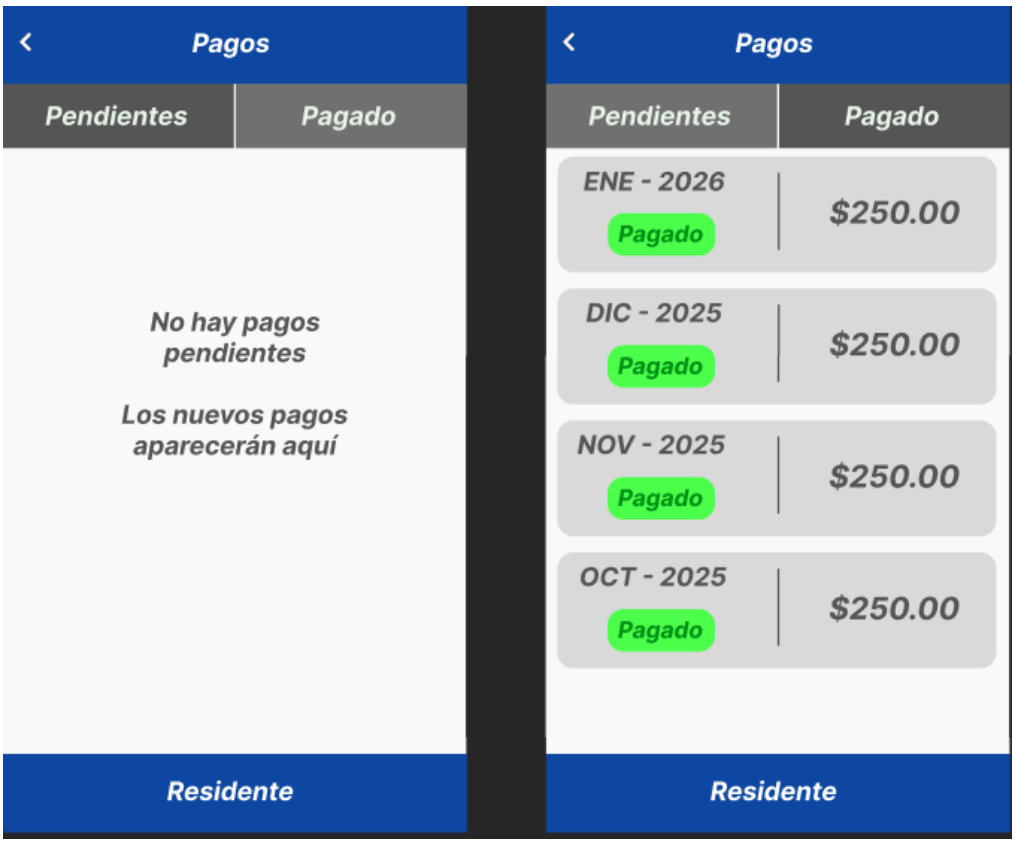
```
        data.put("usuario", email);
        data.put("fecha", System.currentTimeMillis());
        db.collection("quejas").add(data)
                .addOnSuccessListener(doc -> {
                    Toast.makeText(this, "¡Enviado con éxito!",
Toast.LENGTH_SHORT).show();
                    finish();
                });
    }
}
```

PaymentsActivity.java

Esta clase pertenece al paquete **ui/user** y es como el "estado de cuenta" personal del residente. Su función es organizar los recibos del mantenimiento para que no haya confusiones: divide todo en dos pestañas (Pendientes y Pagados) para que sepas exactamente qué debes y qué ya se cubrió. Al usar los colores claros y bordes verdes de la app del guardia, el historial se siente limpio y profesional, dándole al usuario la confianza de que sus pagos están registrados correctamente en la base de datos.
CODIGO:

```
public class PaymentsActivity extends AppCompatActivity {;

  private TextView textViewEmpty;

  @Override

  protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_payments);

    db = FirebaseFirestore.getInstance();

    paymentList = new ArrayList<>();

    adapter = new PaymentAdapter(paymentList);

    recyclerView = findViewById(R.id.recyclerViewPayments);

    textViewEmpty = findViewById(R.id.textViewEmpty);

    toggleGroup = findViewById(R.id.toggleGroupPayments);

    recyclerView.setLayoutManager(new LinearLayoutManager(this));

    recyclerView.setAdapter(adapter);

    toggleGroup.addOnButtonCheckedListener((group, checkedId, isChecked) -> {

      if (isChecked) {

        if (checkedId == R.id.btnPendientes) loadPayments("pendiente");

        else if (checkedId == R.id.btnPagado) loadPayments("pagado");

      }

    });

    toggleGroup.check(R.id.btnPagado);

  }

  private void loadPayments(String status) {

    String userId = FirebaseAuth.getInstance().getUid();
```

```
db.collection("pagos")
    .whereEqualTo("userId", userId)
    .whereEqualTo("status", status)
    .orderBy("timestamp", Query.Direction.DESCENDING)
    .get()
    .addOnSuccessListener(querySnapshots -> {
      paymentList.clear();
      for (DocumentSnapshot doc : querySnapshots) {
        paymentList.add(doc.toObject(Payment.class));
      }
      adapter.notifyDataSetChanged();
      if (paymentList.isEmpty()) {
        textViewEmpty.setVisibility(View.VISIBLE);
        textViewEmpty.setText(status.equals("pendiente") ?
          "No hay pagos pendientes" : "No hay historial de pagos");
      } else {
        textViewEmpty.setVisibility(View.GONE);
      }
    });
  }
}
```



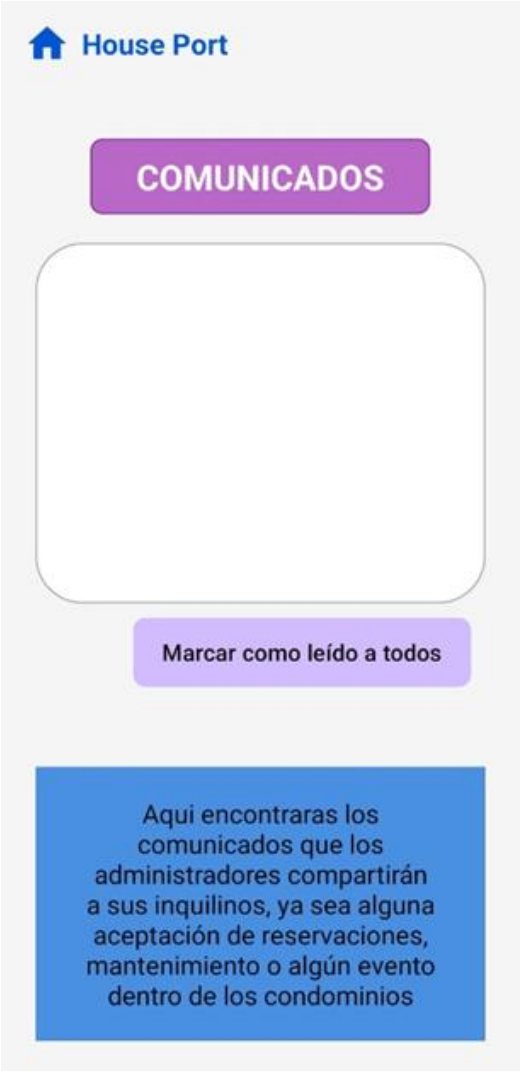
AnnouncementsActivity.java

Esta clase pertenece al paquete ui/user y se encarga de mostrar la sección de noticias y comunicados oficiales para los residentes. Su objetivo es permitir que cualquier usuario consulte la lista de avisos publicados por la administración, presentándolos en un orden cronológico para que lo más reciente aparezca primero. Esto contribuye a mejorar la transparencia y la comunicación dentro del condominio, asegurando que todos los vecinos estén al tanto de las novedades importantes desde la comodidad de su aplicación.

CODIGO:

```
public class AnnouncementsActivity extends AppCompatActivity {
    private RecyclerView recyclerView;
    private AnnouncementAdapter adapter;
    private List<Map<String, Object>> announcementList;
    private FirebaseFirestore db;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_announcements);
        db = FirebaseFirestore.getInstance();
        recyclerView = findViewById(R.id.recyclerViewAnnouncements);
        announcementList = new ArrayList<>();
        adapter = new AnnouncementAdapter(announcementList);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        recyclerView.setAdapter(adapter);

        findViewById(R.id.buttonBack).setOnClickListener(v -> finish());
        loadAnnouncements();
    }
    private void loadAnnouncements() {
        db.collection("avisos")
            .orderBy("fecha", Query.Direction.DESCENDING)
            .get()
```

```
            .addOnSuccessListener(queryDocumentSnapshots -> {
                announcementList.clear();
                for (DocumentSnapshot document : queryDocumentSnapshots) {
                    announcementList.add(document.getData());
                }
                adapter.notifyDataSetChanged();
            })
            .addOnFailureListener(e -> Toast.makeText(this, "Error al cargar",
Toast.LENGTH_SHORT).show());
    }
}
```



ReservationsActivity.java

Esta clase pertenece al paquete ui/user y se encarga de gestionar la selección de espacios dentro del condominio. Su objetivo es proporcionar una interfaz sencilla mediante menús desplegables (Spinners) para que el residente elija la ubicación exacta (edificio, piso y habitación) que desea reservar. Al utilizar adaptadores personalizados para los Spinners, la aplicación mantiene una estética coherente con el resto del diseño, facilitando un proceso de selección rápido y sin errores de escritura.

```
public class ReservationsActivity extends AppCompatActivity {


    private Spinner spinnerEdificio, spinnerPiso, spinnerHabitacion;


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_reservations);
        spinnerEdificio = findViewById(R.id.spinnerEdificio);
        spinnerPiso = findViewById(R.id.spinnerPiso);
        spinnerHabitacion = findViewById(R.id.spinnerHabitacion);
        setupSpinner(spinnerEdificio, new String[]{"1", "2", "3"});
        setupSpinner(spinnerPiso, new String[]{"1", "2", "3", "4"});
        setupSpinner(spinnerHabitacion, new String[]{"1", "2", "3", "4", "5"});


        findViewById(R.id.buttonReservar).setOnClickListener(v -> {
            String seleccion = "Reserva: Ed " + spinnerEdificio.getSelectedItem() +
                               ", P " + spinnerPiso.getSelectedItem() +
                               ", Hab " + spinnerHabitacion.getSelectedItem();
            Toast.makeText(this, seleccion, Toast.LENGTH_LONG).show();
        });
    }


    private void setupSpinner(Spinner spinner, String[] options) {
        ArrayAdapter<String> adapter = new ArrayAdapter<>(this, R.layout.spinner_item,
options);
```

```
        adapter.setDropDownViewResource(R.layout.spinner_dropdown_item);
        spinner.setAdapter(adapter);
    }
}
```

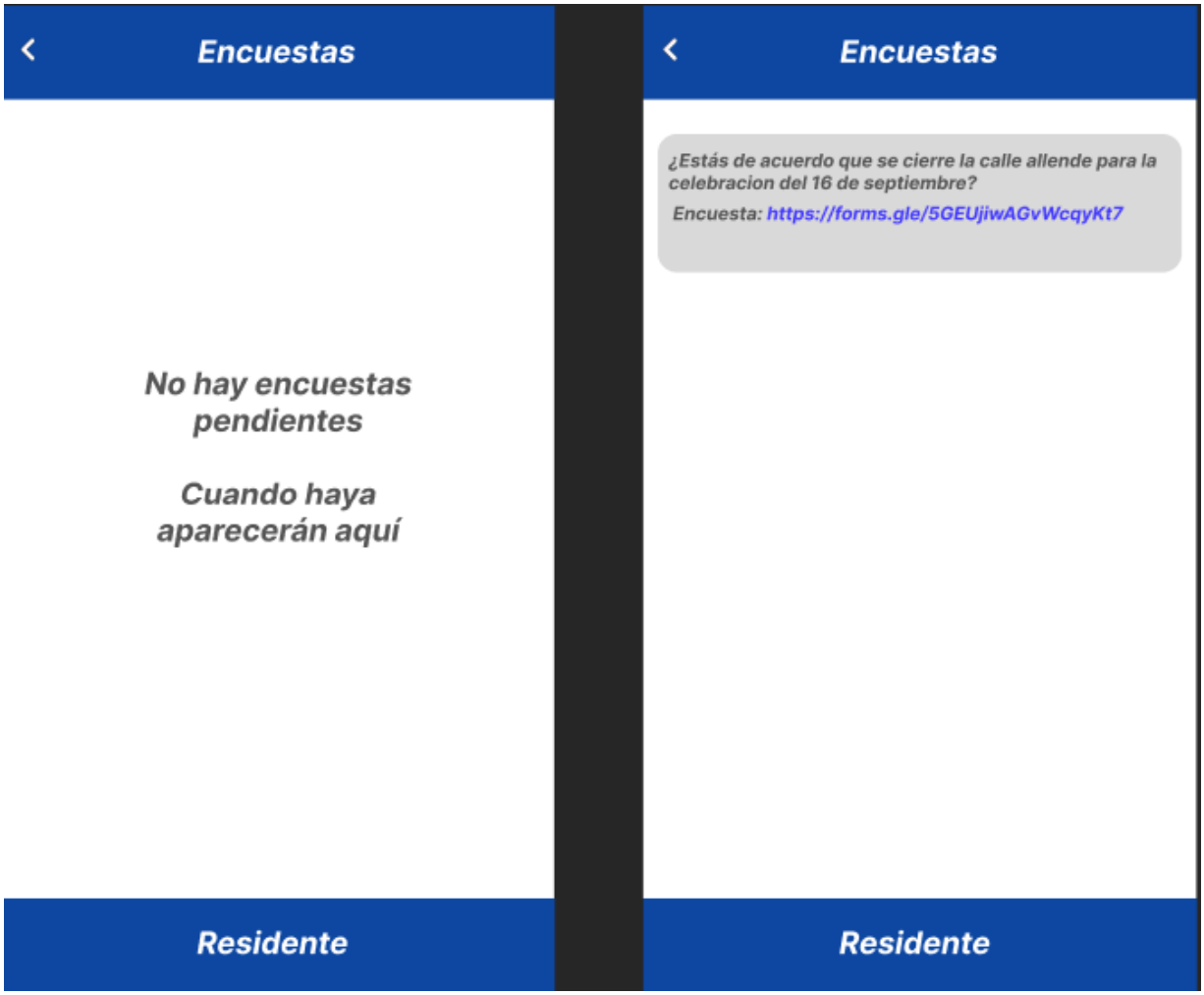
SurveysActivity.java

Esta clase pertenece al paquete **ui/user** y funciona como la "voz del residente" dentro del condominio. Su objetivo es listar las consultas o preguntas que la administración lanza para tomar decisiones juntos. Si no hay nada nuevo que votar, te avisa con un mensaje sencillo, pero cuando hay una encuesta activa, te muestra una tarjeta elegante con el enlace directo. Es la herramienta clave para que la comunidad participe y se sienta escuchada sin tener que ir a juntas presenciales.

CODIGO:

```
public class SurveysActivity extends AppCompatActivity {

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_surveys);

        db = FirebaseFirestore.getInstance();

        surveyList = new ArrayList<>();

        adapter = new SurveyAdapter(surveyList);

        recyclerView = findViewById(R.id.recyclerViewSurveys);

        textViewEmpty = findViewById(R.id.textViewEmptySurveys);

        recyclerView.setLayoutManager(new LinearLayoutManager(this));

        recyclerView.setAdapter(adapter);

        findViewById(R.id.buttonBack).setOnClickListener(v -> finish());

        loadSurveys();

    }

    private void loadSurveys() {

        db.collection("encuestas")

                .orderBy("timestamp", Query.Direction.DESCENDING)

                .get()

                .addOnSuccessListener(querySnapshots -> {

                    surveyList.clear();

                    for (DocumentSnapshot doc : querySnapshots) {

                        surveyList.add(doc.toObject(Survey.class));

                    }
```

```
                adapter.notifyDataSetChanged();
                if (surveyList.isEmpty()) {
                    textViewEmpty.setVisibility(View.VISIBLE);
                    textViewEmpty.setText("No hay encuestas pendientes\nCuando
haya aparecerán aquí");
                } else {
                    textViewEmpty.setVisibility(View.GONE);
                }
            });
    }
}
```



AccessControlActivity.java

Esta clase gestiona la funcionalidad de control de accesos al condominio. Su objetivo es permitir generar nuevos códigos QR y visualizar el historial de entradas de forma rápida, ayudando a mantener un control de quién ingresa a las instalaciones en tiempo real. Al estar vinculada directamente con la base de datos, asegura que el usuario siempre vea su información actualizada, lo que contribuye a mejorar la seguridad y la organización del acceso para los residentes.

CODIGO:

```
public class AccessControlActivity extends AppCompatActivity {


    private RecyclerView recyclerViewHistory;

    private AccessAdapter adapter;

    private List<AccessRecord> accessHistoryList;

    private FirebaseAuth mAuth;

    private FirebaseFirestore db;


    @Override


    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_access_control);

        mAuth = FirebaseAuth.getInstance();

        db = FirebaseFirestore.getInstance();

        // Configuración de RecyclerView y botones

        recyclerViewHistory = findViewById(R.id.recyclerViewAccessHistory);

        recyclerViewHistory.setLayoutManager(new LinearLayoutManager(this));

        adapter = new AccessAdapter(accessHistoryList);

        recyclerViewHistory.setAdapter(adapter);

        findViewById(R.id.buttonGenerateQR).setOnClickListener(v ->
generateNewAccessQR());

        loadAccessHistory();

    }
```

```
    private void loadAccessHistory() {
        FirebaseUser user = mAuth.getCurrentUser();
        if (user == null) return;
        db.collection("registro_de_entrada")
                .whereEqualTo("userId", user.getUid())
                .orderBy("timestamp", Query.Direction.DESCENDING)
                .addSnapshotListener((value, error) -> {
                    if (value != null) {
                      accessHistoryList.clear();
                      for (DocumentSnapshot doc : value.getDocuments()) {
                         AccessRecord record = doc.toObject(AccessRecord.class);
                         if (record != null) accessHistoryList.add(record);
                      }
                      adapter.notifyDataSetChanged();
                      if (!accessHistoryList.isEmpty())
displayLatestQR(accessHistoryList.get(0));
                    }
                });


    }
    private void generateNewAccessQR() {
        FirebaseUser user = mAuth.getCurrentUser();
        String key = generateRandomKey(8);
        AccessRecord record = new AccessRecord(user.getUid(), user.getEmail(), key,
"2026-03-08", "21:40");
        record.status = "GENERADO";
        db.collection("registro_de_entrada").add(record)
                .addOnSuccessListener(doc -> Toast.makeText(this, "¡QR generado!",
Toast.LENGTH_SHORT).show());
    }
    private void displayLatestQR(AccessRecord record) {
        try {
```

```
            Bitmap bitmap = encodeAsBitmap("UID:" + record.userId + "|KEY:" +
record.key);
            ((ImageView)findViewById(R.id.imageViewQR)).setImageBitmap(bitmap);
        } catch (WriterException e) { e.printStackTrace(); }
    }
}
```